LOGPRESS / EXPERIMENTAL RESULTS

Attention 위치 편향 검증 실험

원본 Ranking · 입력 순서 통제 · Layer/Head · Token Attention

Qwen2.5-3B-Instruct | Thunderbird | 2026.11.02

연구 질문과 실험 구성

ERROR/WARN Anchor 가 첫 줄을 1 위로 고르는 이유는 무엇인가 ?

기본 실행

3 × 10

3 개 Window, 각 10 줄

Query Anchor

4 개

ERROR 3 개 · WARN 1 개

Layer/Head

36 × 16

Anchor 당 576 개 조합

측정 전략

① 원본 순위 확인 ② Prefix 순서 변경 ③ Layer/Head 집계 ④ 첫 Key Token 분해

실험 1 | 원본 Anchor Ranking

원본 4 개 Anchor 모두 첫 번째 로그가 Ranking 1 위

Anchor Line	첫 로그 ID	1 위 Attention	결과
ERROR 1311	1302	0.44178	첫 줄 1 위
ERROR 1360	1351	0.48838	첫 줄 1 위
ERROR 1384	1378	0.39633	첫 줄 1 위
WARN 1387	1378	0.35043	첫 줄 1 위

해석 : 첫 위치로 점수가 집중되는 현상 관찰 . 이 결과만으로 위치 편향의 원인을 확정할 수는 없음 .

실험 1 · Original Anchor Ranking 은 무엇을 검사했나 ?

원본 로그 순서를 유지했을 때 , ERROR/WARN Query 가 과거 로그 중 어디에 주목하는지 측정

01 데이터 선택

Thunderbird 로그 3 개 Window × 각 10 줄

02 Query Anchor 선택

level 이 ERROR 또는 WARN 인 로그를 Query 로 사용

03 Attention 변환

Qwen Token Attention → Key SUM, Query MEAN 으로 Line Attention 생성

04 Ranking 산출

Anchor 이전 로그만 후보로 두고 Attention 내림차순 정렬

고정 / 비교 조건 입력 로그의 순서 · 내용 ·Anchor 는 원본 그대로 유지

해석 4 개 Anchor 에서 모두 첫 로그가 1 위였는지 확인

주의 이 결과만으로 첫 위치 때문인지 로그 내용 때문인지 구분할 수 없음 .

실험 2 | Prefix 순서 변경

동일 Anchor · 동일 후보 집합에서 "첫 위치"만 바꾸면?

Anchor	Original 1위	Reverse 1위	Rotate 1위
ERROR 1311	1302 (0.442)	1310 (0.436)	1306 (0.496)
ERROR 1360	1351 (0.488)	1359 (0.473)	1355 (0.481)
WARN 1387	1378 (0.350)	1386 (0.402)	1382 (0.343)

새로운 첫 줄이 1위

9 / 9

3개 Anchor × 3개 조건

제외 기준

ERROR 1384는 순서 변경 후 후보 수가 달라져 공정 비교에서 제외

실험 2 · Reverse / Rotate 는 무엇을 바꿨나 ?

로그 의미보다 입력 위치가 Ranking 에 영향을 주는지 살피는 순서 개입 실험

01 Original

기존 10 줄 순서 유지

02 Reverse

Anchor 는 마지막에 고정 , 앞쪽 9 줄만 역순으로 재배열

03 Rotate

Anchor 는 마지막에 고정 , 앞쪽 9 줄을 4 칸 순환 이동

04 비교

조건별 첫 번째 로그 ID 와 Rank 1/Score 비교

고정 / 비교 조건 가능한 비교에서는 Anchor 동일 , Prefix 의 로그 집합 동일 ; Prefix 의 위치만 변경

해석 비교 가능한 3 개 Anchor × 3 조건 = 9/9 에서 새 첫 로그가 1 위

주의 순서를 바꾸면 Causal Attention 이 보는 문맥도 바뀜 . ERROR 1384 는 후보 수가 달라 제외 .

실험 3 | 첫 로그의 Attention 은 어디에서 오는가 ?

첫 로그 전체 Attention 의 92~94% 가 첫 Token 하나에서 발생

Anchor	첫 Key Token	첫 로그 나머지	첫 로그 전체	첫 Token 비중
1311	0.41418	0.02760	0.44179	93.75%
1360	0.45292	0.03545	0.48838	92.74%
1384	0.36677	0.02957	0.39634	92.54%
1387	0.33068	0.01975	0.35043	94.36%

정의 : 첫 Token 비중 = 첫 Token Attention ÷ 첫 로그 전체 Attention

전체 시퀀스에 대한 점유율과 구분해야 함

실험 3 · Token Attention 을 왜 다시 측정했나 ?

Line Attention 이 높게 보이는 현상이 한 Token 에서 생겼는지 직접 분해

01 기존 결과의 한계

Line Attention .pt 에는 Token 별 원본 가중치가 없음

02 원본 행렬 취득

model(..., output_attentions=True) 에서 각 Layer/Head 의 [T × T] 추출

03 Anchor Query 평균

같은 Anchor 로그에 속한 Query Token 들에 대해 평균

04 Key 쪽 점수 분리

첫 로그의 첫 Key Token / 나머지 Key Token / 다른 로그로 나눠 합산

고정 / 비교 조건 **Anchor 별 Layer 36 × Head 16 = 576 개 값 저장 ; 4 개 Anchor 총 2,304 행**

해석 첫 로그 Attention 의 약 92~94% 가 첫 Token 하나에서 나온다는 점 확인

주의 첫 Token 이 실제 어떤 문자열인지는 별도 확인 필요 . Attention 은 RCA 정답이 아님 .

실험 4 | Layer/Head 수준의 첫 Token 집중

첫 Token 점수가 같은 로그의 나머지 Token 합보다 큰 조합



첫 줄 집중은 전체 평균만의 산물이 아니며 , 여러 Layer/Head 에 걸쳐 관찰됨

실험 4 · Layer/Head 분석은 무엇을 비교했나 ?

전체 평균에 의한 착시인지 , 모델 내부 여러 Layer/Head 에서 나타나는지 확인

01 분석 단위

하나의 Anchor 에 대해 36 개 Layer × 16 개 Head 검사

02 편향 판정

첫 Key Token 의 Attention > 같은 첫 로그의 나머지 Token 합

03 비율 계산

조건을 만족한 조합 수 ÷ 576 × 100 (%)

04 Layer 추세

같은 Layer 안의 Head 16 개를 평균해 앞 / 중간 / 끝 Layer 비교

고정 / 비교 조건 같은 Anchor 와 같은 원본 Window 에서 Layer/Head 인덱스만 비교

해석 4 개 Anchor 에서 509~525/576 개 조합이 첫 Token 우세

주의 전체 Head 가 동일하게 행동하는 것은 아니며 Attention 집중과 실제 근거성은 별개 .

Layer 별 변화 | ERROR 1311

중간 Layer 에서 첫 Token 집중이 강하고 , 처음 · 마지막 Layer 에서는 약해짐



측정 : Anchor 1311 의 각 Layer 에서 16 개 Head 평균

결론 및 이번 주 계획

실험 결론

첫 Token 집중이 첫 로그의 Line Attention 을 높여 Ranking 1 위에 영향을 줌

현재 확인된 한계

위치 편향은 확인했지만 , Anchor Attention 이 실제 Evidence 를 잘 찾는지는 미검증이다 .
첫 번째 Token 제외나 위치 편향 보정은 탐지 성능 평가 이후 검토한다 .

이번 주 할 일 · 코드 설계

① Window 추출 코드 설계 : 실제 Evidence 가 첫 줄이 아닌 10 줄 Window 선정
② 평가지표 코드 설계

주간 요약

LogPress 이번 주 진행 요약

Part A는 4개 Baseline의 공통 평가 연결을 완료했고, Part B는 Nezha 원본 데이터를 내려받아 구조 확인을 시작했습니다.

Part A

Baseline 4종 통합 완료
LineScore 평가 · 36 Layer 추적

Part B

Nezha 공식 원본 확보
CSV · fault_list 위치 확인

다음 단계

원인 정보와 로그 줄 연결
필요 시 Part C 기술 지원

이번 주 핵심 결론

- ObservedAttention · SnapKV · StreamingLLM · H2O를 같은 코드로 평가할 수 있습니다.
- Thunderbird 65줄 샘플에서 압축률 0.5 / 0.7 / 0.9 실험을 확인했습니다.
- Nezha의 공식 장애 정보를 실제 원인·증거 로그 줄에 연결하는 작업이 남았습니다.

Part A 진행 상황

완료한 내용

- ObservedAttention · SnapKV · StreamingLLM · H2O를 공통 Evaluator에 연결했습니다.
- Qwen2.5-3B 모델의 36개 Layer와 KV Head별 생존 토큰을 추적했습니다.
- StreamingLLM의 Sink/Recent, H2O의 Heavy Hitter/Recent 동작을 검증했습니다.

공통 평가 흐름

원본 토큰 위치
KV 생존 토큰

→

로그 라인 점수
LineScore

→

Top-K 선택
라인 예산

→

정답 비교
GT 보존율

공통 코드

- line_mapping.py : 토큰 ↔ 로그 라인 연결
- line_score.py : Layer × KV Head 생존율 평균
- evaluator.py : Top-K 및 정답 보존율 계산

이번 실험 조건

- Qwen2.5-3B-Instruct · 36 Layers · KV Heads 2
- Thunderbird 로그 65줄 · 이상 GT 17줄
- KV 압축률 0.5 / 0.7 / 0.9

Baseline 평가 결과

Thunderbird 단일 샘플 · Anomaly Line Retention

동일한 로그 65줄 / 이상 로그 17줄 / LineScore 기반 Top-K

Baseline	압축률 0.5	압축률 0.7	압축률 0.9
StreamingLLM	**8 / 17 (47.06%)**	**5 / 17 (29.41%)**	**0 / 17 (0%)**
H2O (Prefill 변형)	7 / 17 (41.18%)	4 / 17 (23.53%)	1 / 17 (5.88%)
ObservedAttention	6 / 17 (35.29%)	4 / 17 (23.53%)	0 / 17 (0%)
SnapKV	6 / 17 (35.29%)	2 / 17 (11.76%)	1 / 17 (5.88%)

이번 결과에서 확인한 점

- StreamingLLM이 압축률 0.5와 0.7에서 가장 높았지만, 한 샘플의 결과입니다.
- 이번 정답 17줄은 Thunderbird의 이상 로그이며 RCA 원인·증거 GT는 아닙니다.
- H2O는 원본의 순차적 eviction이 아닌 prefill attention 기반 변형입니다.

평가상 주의 Top-K로 선택한 라인은 실제 KV에 완전한 문장이 남았다는 뜻은 아닙니다. KV 압축률과 line budget 대응도 최종 검토가 필요합니다.

Part B 진행 상황 및 다음 작업

확인한 Nezha 원본 구조

- 공식 Nezha 저장소를 별도로 clone 완료
- rca_data/2022-08-22/ 폴더 확인
- log/ · metric/ · trace/ · traceid/ 존재
- 시간대별 CSV와 fault_list.json 확인

현재 상태

- 03_53_log.csv 등 로그 파일 확인
- 2022-08-22-fault_list.json 위치 확인
- 아직 장애 목록 내용과 로그의 연결은 미분석
- 실제 원인·증거 라인 GT는 구축 전 단계

Part B 앞으로의 작업

- 1. fault_list.json을 열어 장애 시각 · 대상 서비스 · 주입 유형 확인
- 2. 장애 사례 하나를 골라 관련 CSV 로그와 연결
- 3. 전체 사례에서 원인·증거 라인 후보를 자동으로 찾기
- 4. success / ambiguous / not_found로 구분하고 일부 직접 검수
- 5. 개발용 / 평가용 데이터와 loader를 정리해 Part D에 전달

요약 A는 Baseline 4종의 1차 평가 연결을 마쳤습니다. B는 Nezha 원본을 확보했고, 다음에는 공식 장애 정보와 실제 로그 줄을 연결합니다.

2층 평가 (효율 / 비용) — 보충 설계

GPU 피크 · 베이스라인 분류 · forward 비용 처리 · 파레토 '같음' 판정

1. 개념

GPU 피크란?

- **추론 중 GPU 메모리를 가장 많이 쓴 그 순간의 사용량** (peak = 최고점)
- 코드로는 `torch.cuda.max_memory_allocated()` — 돌리는 동안 할당된 메모리의 **최댓값**
- GPU는 그 순간 못 버티면 터짐(OOM) → "이 GPU에 올릴 수 있냐"는 **평균이 아니라 피크**로 결정됨
- 우리 압축기가 KV를 줄이면 이 피크가 내려감 → 같은 GPU에 더 긴 로그 / 더 많은 요청 수용

> ⚠️ 피크 실측값엔 KV만이 아니라 **모델 가중치·활성화까지 섞여서** 잡힌다. → 순수 KV 절감은 **이론 계산값**으로 보고, 피크 실측은 "실제로도 안 터지고 줄더라" 확인용으로 사용.

2. 베이스라인

무엇으로 KV를 남기나

베이스라인	KV 선택 기준	어텐션 점수	동적/정적	eager forward
SnapKV	최근 쿼리의 어텐션 점수 → 중요한 prefix 토큰 top-k	✅ 읽음	동적	필요
H2O	누적 어텐션 점수 높은 토큰(heavy hitter) + 최근	✅ 읽음	동적	필요
ObservedAttention	관측된 어텐션 가중치로 점수화 후 제거	✅ 읽음	동적	필요
StreamingLLM	맨 앞 sink 토큰 + 최근 윈도우 (**위치 고정**)	❌ 안 읽음	정적	불필요
random KV	무작위 제거	❌	정적(무작위)	불필요
level filter (우리 비교군)	로그 레벨(ERROR/WARN 등)	❌	정적	불필요
우리 압축기	로그 신호(레벨·희귀도·위치) + 어텐션	✅ 읽음	동적	필요

2. 베이스라인

두 부류로 갈림

어텐션 점수 기반 (동적)

SnapKV · H2O · ObservedAttention · 우리

런타임에 어텐션 점수를 읽어 남길 KV를 고름
→ 점수를 메모리에 띄우는 **eager 어텐션 필요**

위치·규칙 기반 (정적)

StreamingLLM · random KV · level filter

고정 규칙으로 남김 (위치·무작위·레벨)
→ 어텐션 점수 안 읽음 → **eager 어텐션 불필요**

용어 주의: "어텐션 기반"은 두 뜻으로 섞임 — 넓게(어텐션 위에 세움) vs 좁게(런타임에 점수 읽음). forward 비용에선 **좁은 뜻(점수 읽음)**이 중요 → "점수 기반 / 위치·규칙 기반"으로 표기.

비교 상대별 방법

비교 상대	방법
점수 기반 SnapKV · H2O · ObservedAttention	forward가 양쪽 공통 → **소거 지름길 OK** (또는 끝까지 실측해도 자동 상쇄)
위치·규칙 기반 StreamingLLM · random KV · level filter	소거 지름길 쓰지 말 것. 끝까지 **실측값 그대로** 사용 → 우리 eager forward 오버헤드가 그 숫자에 정직하게 잡히게 둠

원칙

"끝까지 실측하면 공통분은 저절로 상쇄된다."
점수 기반끼리만 'forward 빼자' 지름길을 쓰고, 위치·규칙 기반엔 실측값을 그대로 둔다 (빼주면 우리에게 유리하게 조작한 셈).

4. § 5 합격 기준

파레토 '같음' 판정 (오차율 처리)

오차율은 숫자로 정하지 않는다. "5% 이내면 같음" 같은 임의 기준 대신, 여러 번 재서 흔들림(CI)이 **0을 걸치는지**로 판정.

- ① **측정** — 차이값(우리 − baseline)을 표본으로 모은다. Nezha 장애 케이스별로 LogPress/baseline 보존율 차이 → baseline마다 38개 케이스 = CI 1개
- ② **계산** — 그 표본으로 **부트스트랩 95% 신뢰구간(CI)**을 뽑는다. CI = 측정값들로 계산한 "범위"
- ③ **판정** — CI가 0을 기준으로 어디 있는지 본다 ↓

CI 위치	판정
CI 전체 > 0	이김
CI 전체 < 0	짐
CI가 0을 포함	**같음**

파레토 합격

두 축(보존율·메모리) 모두 '짐'이 없고, 최소 한 축에서 '이김'.

4. § 5 합격 기준

표본이 뭔지 + 정할 숫자

- **보존율 축:** 표본 = **케이스들** (케이스마다 보존율 차이 → 그 묶음으로 CI)
- **메모리·속도 축:** 표본 = **반복 측정 횟수** (같은 조건 N번 재서 그 묶음으로 CI)
- ※ 베이스라인 5개는 표본이 아니라 **5번의 1:1 비교** → 각자 CI. 비교가 많으므로 '이김' 선언엔 **BH-FDR 보정** (1층과 동일).

숫자	뭐냐	값
케이스 수 (Nezha 정제 필요)	보존율 축 표본	20~30+ (많을수록 CI 좁아짐. 2개면 다 '같음')
측정 반복 N	메모리·속도 축 표본	속도 10회↑ / 메모리 3회 (워밍업 제외)
부트스트랩 B	CI 계산 내부 재표집	5,000 (코드 상수, 1층과 동일)

핵심: 오차율을 사람이 정하지 않고 **데이터의 흔들림이 '같음'의 경계를 정한다.** 값들은 관례 수준 — 중요한 건 **실험 전에 정해서 프리즈** (결과 보고 바꾸면 어뷰징).

5. 측정

CI 뽑는 순서 — 재표집

0. 입력 — 각 케이스에서 우리 보존율 − baseline 보존율 → 길이 38 배열

```
diff = [+0.05, 0.00, −0.03, +0.02, ...]   # 길이 38
```

재표집 = "같은 칸을 복원추출로 다시 채움" (중복 OK, 누락 OK) — 설명용 6칸

```
resample #1 = [+0.05, +0.05, 0.00, +0.02, −0.01, +0.04]   mean = +0.0250
resample #2 = [−0.03, 0.00, +0.02, +0.02, +0.04, +0.05]   mean = +0.0167
resample #3 = [+0.04, +0.05, +0.02, 0.00, −0.03, +0.02]   mean = +0.0167
  ... 5,000번 반복 → 평균 5,000개
```

원본 38칸을 복원추출로 다시 채워 **평균 1개** → 5,000번 = 평균 5,000개. 이 5,000개를 정렬해 **2.5% · 97.5% 자르면 95% CI.** (새 데이터를 더 재는 게 아니라, 있는 걸 섞어 "평균이 얼마나 흔들리나"를 보는 것)

5. 측정

"CI가 0을 기준으로"의 뜻

- CI 안의 값은 전부 **차이값 = (우리 − baseline)** → 그래서 **0 = 차이 없음 = 같음**이 기준점
- 값이 **양수** = 우리가 더 높다 / **0** = 같다 / **음수** = 우리가 더 낮다

```
[+1.2, +3.5]   범위가 통째로 0보다 큼    → 어떤 값이든 우리가 높음  → 이김
[−3.5, −1.2]   범위가 통째로 0보다 작음  → 어떤 값이든 우리가 낮음  → 짐
[−1.0, +2.0]   범위가 0을 걸침           → +일 수도 0일 수도 −일 수도 → 같음
```

마지막이 '같음'인 이유: 진짜 차이가 +2일 수도, 0일 수도, −1일 수도 있어서 **"우리가 확실히 다르다"고 말할 수 없음** → 사실상 같다고 본다.

요약 — 차이값이 "우리 − baseline"이라 **0이 '같음' 지점.** CI(차이의 범위)가 0 위 / 0 아래 / 0 걸침 → 이김 / 짐 / 같음.